Spectral Graph Partitioning Tool

Ivan Zaluzhnyy

Spring 2026

Abstract

This project is a Java Swing application that takes an undirected graph and partitions it into two groups. Internally, the program represents the graph as an adjacency matrix, builds the degree matrix and the Laplacian from that, and then hands the Laplacian off to Apache Commons Math for the eigendecomposition. The Fiedler vector (the eigenvector for the second-smallest eigenvalue) is what the partitioning is actually based on. The user can supply the input graph in three ways: by loading an adjacency matrix from a text file, by generating a random graph, or by drawing one directly on the canvas, and after a successful partition, the two resulting groups are pulled apart into separate regions of the results canvas, with the cut edges drawn in a different color. The program does not always agree to partition a graph: if the graph is disconnected, or too densely connected, or if the best available cut is still not clean enough under the scoring rule used here, the program refuses and explains why.

1. Introduction and Motivation

A graph is a set of points (vertices) with some lines (edges) drawn between them. Even though the structure is this simple, it turns out that a surprising number of real-world systems can be modeled this way: computer, social or road networks, and even molecules (where atoms are vertices and bonds are edges) are all examples in different contexts. Once a system has been written down as a graph, one of the natural questions to ask is where its weakest connection is, meaning, if the graph were cut into two pieces, where would the cut do the least damage?

Spectral graph theory provides one way of answering that question. The basic idea is that the graph gets translated into a couple of matrices, and from there it becomes a linear algebra problem – the eigenvalues and eigenvectors of those matrices end up carrying real structural information about the graph itself. The matrix that matters the most for this project is the graph Laplacian. Its second-smallest eigenvalue has a name (the Fiedler value), and the eigenvector that goes with it is called the Fiedler vector. What makes the Fiedler vector useful is that it tends to separate the graph near a natural bottleneck, if there is one.

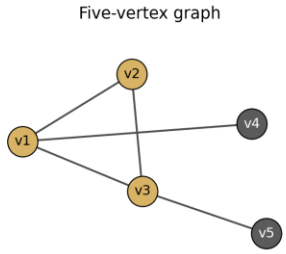
The interest in this topic, for me personally, came out of my earlier exposure to graph methods, molecular modeling and spectral graph theory. What I built here is a small interactive tool whose goal is to let a user see the main idea play out (that a graph can be converted into matrices, and that the Laplacian's eigenvectors can pick out a weak connection between two parts of the graph), rather than to provide a production-grade clustering library. I used the academic sources listed in the references mainly as a way to keep the mathematical explanation and the implementation choices honest.

2. Background: Graphs, Matrices, and the Fiedler Value

2.1 Adjacency, degree, and Laplacian matrices

The goal of matrices is to turn a picture of dots and lines into numerical data that the program can compute with. A graph with n vertices can be stored as an n by n adjacency matrix A. Entry A[i][j] is 1 if vertices i and j share an edge, and 0 otherwise. Because this project uses undirected graphs, the matrix is symmetric.

The degree matrix D is diagonal. Its i-th diagonal entry is the number of edges attached to vertex i. The graph Laplacian is then defined as L = D - A. This is the matrix used by the partitioning algorithm.



Adjacency A

0	1	1	1	0
1	0	1	0	0
1	1	0	0	1
1	0	0	0	0
0	0	1	0	0

Degree D

3	0	0	0	0
0	2	0	0	0
0	0	3	0	0
0	0	0	1	0
0	0	0	0	1

Laplacian L = D - A

3	-1	-1	-1	0
-1	2	-1	0	0
-1	-1	3	0	-1
-1	0	0	1	0
0	0	-1	0	1

Figure 1. Example illustrating standard graph-matrix definitions: a five-vertex graph, its adjacency matrix A, degree matrix D, and Laplacian L = D - A. This and other diagrams were drawn in Draw.io.

2.2 Fiedler value and Fiedler vector

For an undirected graph, the smallest Laplacian eigenvalue is 0. The second-smallest eigenvalue is called the **Fiedler value** or **algebraic connectivity**. The behavior of this value is what makes it useful. When the graph is disconnected, the Fiedler value drops to 0 (or to a number numerically close to 0). When the graph is connected but has a weak spot/bottleneck somewhere, the Fiedler value is small but nonzero. And when the graph is densely connected with no obvious weak spot, the value gets larger. In this project, the program uses the Fiedler value to decide whether the graph is even worth partitioning in the first place, and only proceeds if the value sits in the “weak connection” range.

The Fiedler vector has one component for each vertex. In the clean example below, the graph contains two K4 cliques (tightly connected four-vertex groups, a.k.a complete sub-graphs on four (or other number) of vertices). The signs of the Fiedler-vector components separate those two groups. The bridge edge between the two cliques is exactly where a human would expect the graph to split.

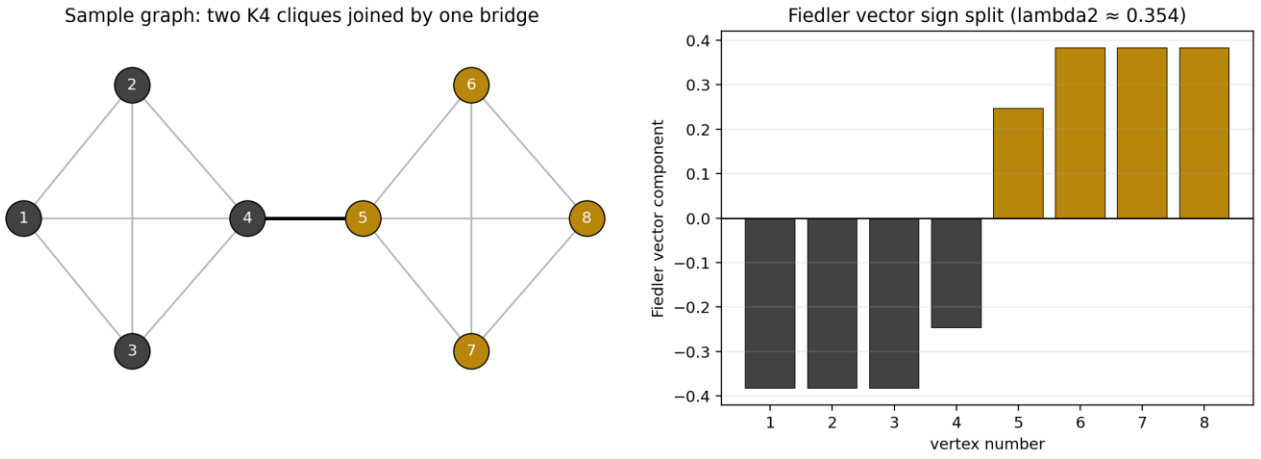


Figure 2. Fiedler-vector sign split for the supplied Sample Graph.txt graph. Values computed using program Java code. Calculations done in Excel. Figures generated in draw.io.

The final program does not depend only on a simple sign split. During testing, I found that a sign split can behave poorly on lopsided graphs. The final version sorts vertices by their Fiedler-vector components and then sweeps through possible split points to find the cut with the best score (discussed separately).

3. Class Design

The application is organized into five classes.

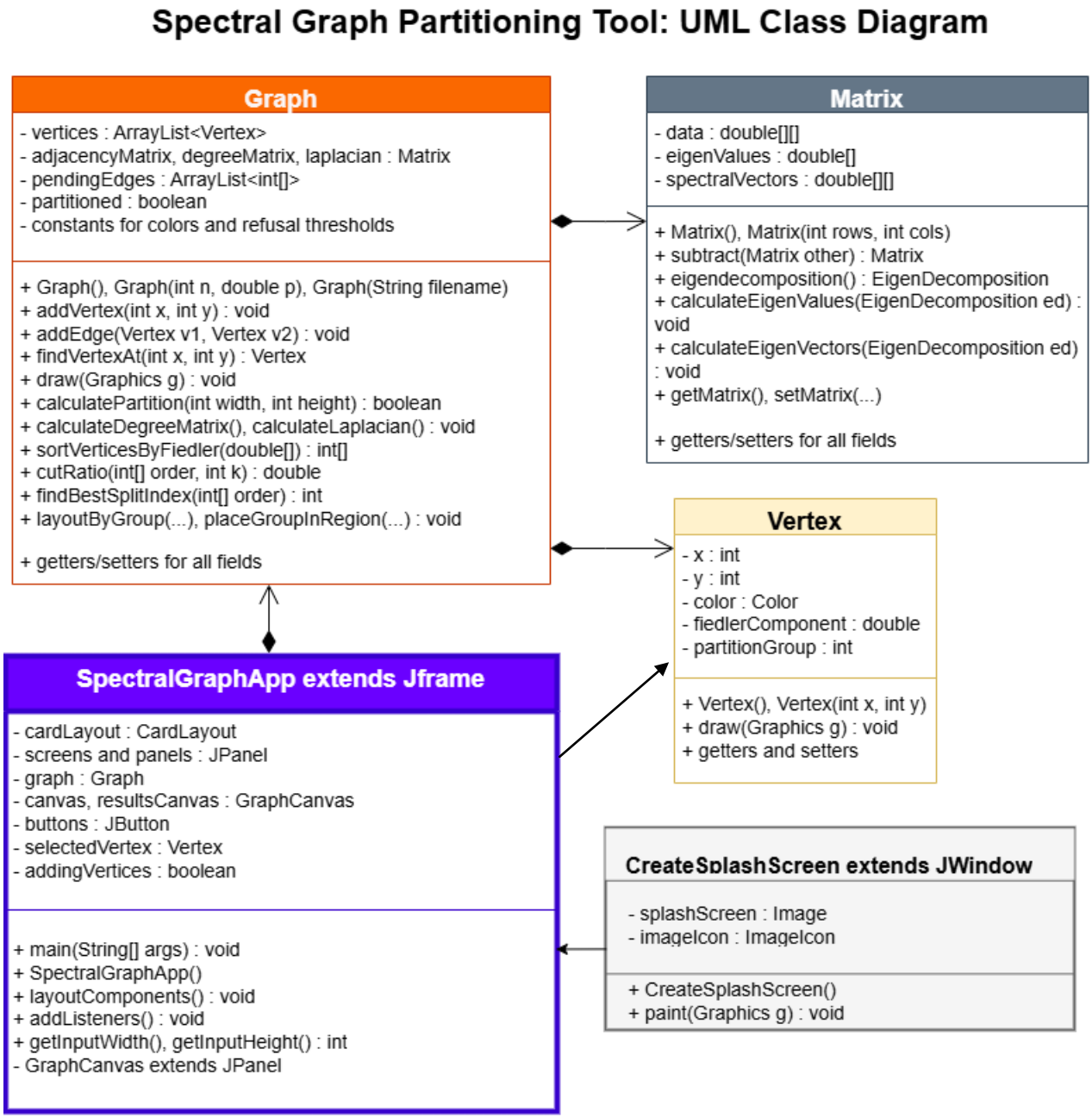


Figure 3. UML class diagram of the application. Created in Draw.io from Java source files. The Graph class owns the graph data and runs the partitioning algorithm; Matrix stores matrix data and delegates eigendecomposition to Apache Commons Math; Vertex stores display and partition data for one vertex; SpectralGraphApp owns the GUI; CreateSplashScreen displays the startup image.

3.1 SpectralGraphApp

SpectralGraphApp is the main GUI class. It extends JFrame, and it owns essentially all of the user-facing pieces (the screens, the buttons, the canvases, and the event listeners). It is purely an input/output interface shell. When the user clicks something, SpectralGraphApp's job is just to figure out what was clicked and then call the right method on whatever Graph object is currently loaded.

Its main pieces of state are the CardLayout, the welcome/input/results panels, the current Graph, the input and results canvases, the buttons, the selectedVertex used while drawing edges, and the addingVertices flag used to switch between vertex mode and edge mode of manual input.

3.2 Graph

Graph is the central model class. It stores the list of Vertex objects, the adjacency matrix, the degree matrix, the Laplacian, the pending edges used in draw mode, and the "partitioned" flag variable. It also defines the color and threshold constants used by the algorithm.

Graph knows how to add vertices and edges, find a clicked vertex, draw the graph, build the degree matrix and Laplacian, sort vertices by Fiedler component, evaluate candidate cuts, choose the best split, and reposition vertices into groups. The public calculatePartition method is the main method of the project. It returns true when the partition succeeds and false when the graph is refused due to lack of clear cut.

Graph draw method handles the visual difference between input mode and results mode: before partitioning it draws plain gray edges, while after partitioning it highlights cut edges in black and within-group edges in light gray.

This class has many methods and cascades. Its method-level and per-line comments, theoretical and design logic sections in this report as well as literature references best explain all details of Graph implementation. It also contains adapted and translated code for the more sophisticated logic, as referenced in code comments and this document.

3.3 Matrix

Matrix is a fairly small class, and most of what it does is just wrap a two-dimensional array of doubles. It supports matrix subtraction to compute L = D - A, and, after the eigendecomposition runs, it also stores the resulting eigenvalues and spectral vectors as instance fields. The Matrix class does not implement the eigendecomposition itself; it converts its internal double[][] into an Apache Commons Math RealMatrix and stores the eigenvalues and Fiedler vector returned by Apache.

3.4 Vertex

The Vertex class models a single vertex in the graph. Each Vertex stores its (x, y) coordinates on the canvas, its current display color, the value of the Fiedler-vector component that gets assigned to it during partitioning, and the partition group it ended up in (0 or 1). It also has a small draw method of its own – the vertex is drawn as a filled circle – which keeps the vertex drawing logic out of the Graph class.

3.5 CreateSplashScreen

CreateSplashScreen is a small startup window that the user sees for a few seconds before the main app comes up. It extends JWindow rather than JFrame (which is what gives a splash screen its borderless appearance), displays the logo image, and then hands control over to SpectralGraphApp. The basic structure of this class is adapted from the Tutorialspoint example listed in the references, but the timing and the way it launches the main app are different in my version.

4. The Partitioning Algorithm

4.1 Building the matrices

When calculatePartition runs, it first makes sure the adjacency matrix exists. For file-loaded and random graphs, the adjacency matrix is already built by the constructor. For hand-drawn graphs, the method builds it from the pendingEdges list.

Next, Graph computes the degree matrix by summing each row of the adjacency matrix with the result placed on the main diagonal of the degree matrix. Then it computes the Laplacian as D - A. The Laplacian is passed to Apache Commons Math for eigendecomposition.

4.2 Refusal conditions

The program does not force every graph into a partition. It refuses three cases.

First, if the Fiedler value is approximately zero, the graph is treated as disconnected. Spectral partitioning is not useful in that case because the graph is already separated. The original plan was to recursively partition graph components until it is completely separated by all possible bottlenecks, but this quickly became a second priority and a possible future addition.

Second, if the Fiedler value is above the dense-graph threshold, the graph is treated as too well connected. The motivation comes from reasoning based on Cheeger inequality (please see references for details on the math): a large second-smallest eigenvalue means the graph does not have a clear bottleneck.

Third, even if the Fiedler value passes the earlier checks, the best available split may still cut too many edges. The program computes a cut ratio and refuses the partition if that score is too high. The two 0.9 thresholds are empirical. They were chosen by testing small graphs and looking for behavior that matched the intended visual demo.

Additional details of implementation and sources for code adaptations and translations from Python (the main language for such exercises) are provided in the class javadoc. It was decided that it was in the best interest of the project to rely on select code adaptations than to compromise the quality of output and user experience.

4.3 Sweep over Fiedler order

The first version of the program tried to split the graph by looking only at the signs of the Fiedler-vector components. That works well for clean textbook examples, where one group has negative values and the other group has positive values. However, in testing I found that a simple sign split can behave poorly on less balanced or less obvious graphs. The final version therefore uses a cut-ratio sweep adapted from Roch's graph-partitioning section and accompanying Python code.

$$\phi(S) = \frac{|E(S, S^c)|}{\min\{|S|, |S^c|\}}$$

Figure. Cut-ratio formula from Roch, Mathematical Methods in Data Science, Section 5.5. The numerator counts edges crossing between the two sides of a cut, while the denominator penalizes cuts where one side is too small.
https://mmids-textbook.github.io/chap05_specgraph/05_partitioning/roch-mmids-specgraph-partitioning.html

Roch defines the cut ratio for a split of the vertex set into two non-empty parts, S and S^C. The numerator counts the number of edges crossing between the two sides of the cut. The denominator is the size of the smaller side. This denominator matters because it penalizes cuts that simply isolate one tiny group. For example, cutting one edge to separate one vertex from seven vertices gives a score of 1 / 1 = 1, while cutting one edge to separate four vertices from four vertices gives a score of 1 / 4 = 0.25. The second cut is better because it breaks the same number of edges but produces a more balanced split.

The program does not test every possible subset S. That would be too large a search problem, since the number of possible subsets grows very quickly as the graph gets bigger. Roch's shortcut is to use the Fiedler vector to reduce the problem to a one-dimensional ordering of the vertices. Each vertex has one Fiedler-vector component. The program sorts the vertices by those component values, from smallest to largest, and then tests each possible split point along that sorted order. This is meaningful because the Fiedler vector comes from the Laplacian, and the Laplacian encodes the connectivity structure of the graph. When the graph has a natural bottleneck, the Fiedler-vector ordering often places vertices from one side of the bottleneck toward one end of the ordering and vertices from the other side toward the other end.

Roch gives the cut-ratio scoring rule that was adapted for scoring each candidate split:

An algorithm: The input is the graph $G = (V, E)$. Let $\mathbf{y}_2 \in \mathbb{R}^n$ be the unit-norm eigenvector of the Laplacian matrix L associated to its second smallest eigenvalue μ_2, i.e., $\mathbf{y}_2$ is the Fiedler vector. There is one entry of $\mathbf{y}_2 = (y_{2,1}, \ldots, y_{2,n})$ for each vertex of G. We use these entries to embed the graph G in $\mathbb{R}$: vertex i is mapped to $y_{2,i}$. Now order the entries $y_{2,\pi(1)}, \ldots, y_{2,\pi(n)}$, where π is a permutation, that is, a re-ordering of $1, \ldots, n$. Specifically, $\pi(1)$ is the vertex corresponding to the smallest entry of $\mathbf{y}_2$, $\pi(2)$ is the second smallest, and so on. We consider only cuts of the form

$$S_k = \{\pi(1), \ldots, \pi(k)\}$$

and we output the cut (S_k, S_k^c) that minimizes the cut ratio

$$\phi(S_k) = \frac{|E(S_k, S_k^c)|}{\min\{k, n-k\}},$$

for the $k \leq n - 1$.

Figure 4. Cut-ratio scoring rule adapted from Roch, Mathematical Methods in Data Science, Section 5.5. The numerator counts edges crossing between the two sides of a proposed cut, while the denominator uses the smaller side of the split to penalize one-sided cuts. Source: https://mmids-textbook.github.io/chap05_specgraph/05_partitioning/roch-mmids-specgraph-partitioning.html

In my code, sortVerticesByFiedler creates an order array. This array does not store new vertices. It stores the original vertex indices, sorted by Fiedler-vector component. The sorting method keeps the original vertex indices but orders them by Fiedler-vector value, which is why later code uses order[i] to get back to the original vertex number.

For example, if:

fiedler = [+0.50, -0.40, +0.45, -0.35, -0.10]

vertices: v0 v1 v2 v3 v4

order = [1, 3, 4, 2, 0]

This means that vertex 1 has the smallest Fiedler component, vertex 3 has the next smallest, and vertex 0 has the largest. The array is a permutation of the original vertex numbers.

For a proposed split point k, the program treats order[0] through order[k] as one side of the cut, and order[k + 1] through order[n - 1] as the other side. The cutRatio(order, k) method then counts how many adjacency-matrix entries cross between those two sides. This is why the code uses adj[order[i]][order[j]] instead of adj[i][j]. The variables i and j are positions in the Fiedler-sorted order, but the adjacency matrix is indexed by the original vertex numbers.

The findBestSplitIndex(order) method performs the sweep. It tries every valid split point from k = 0 through k = n - 2, calls cutRatio(order, k) for each candidate, and remembers the split point with the lowest score. In this Java version, it is a standard loop with two tracker variables: bestPhi, which stores the lowest cut ratio seen so far, and bestK, which stores the split point that produced that score.

This is not a full search over every possible cut. It is a practical spectral shortcut: the Fiedler vector gives a meaningful ordering of the vertices, and the cut-ratio sweep chooses the best split along that ordering. For this project, that was the right compromise. And it did make the app behave better than the sign-split version originally tested on small graphs.

4.4 Group assignment and visual repositioning

After the best split index is found, vertices on one side receive partition group 0 and are colored dark gray. Vertices on the other side receive partition group 1 and are colored dark gold. The draw method then colors cut edges black and within-group edges light gray.

The visual vertex repositioning is not part of the spectral cut algorithm itself. It is a display step. Group 0 is placed in the left half of the results canvas and group 1 in the right half. Within each half, vertices are arranged around a circle using a simplified coordinate layout adapted from JGraphT's CircularLayoutAlgorithm2D. The JGraphT source places vertices evenly around a circle; my simplified version computes the same kind of circular coordinates and stores them directly in Vertex objects.

5. Demonstration

5.1 GUI walkthrough

The welcome screen explains the purpose of the application and lets the user choose among three input modes: load from file, generate random, or draw.

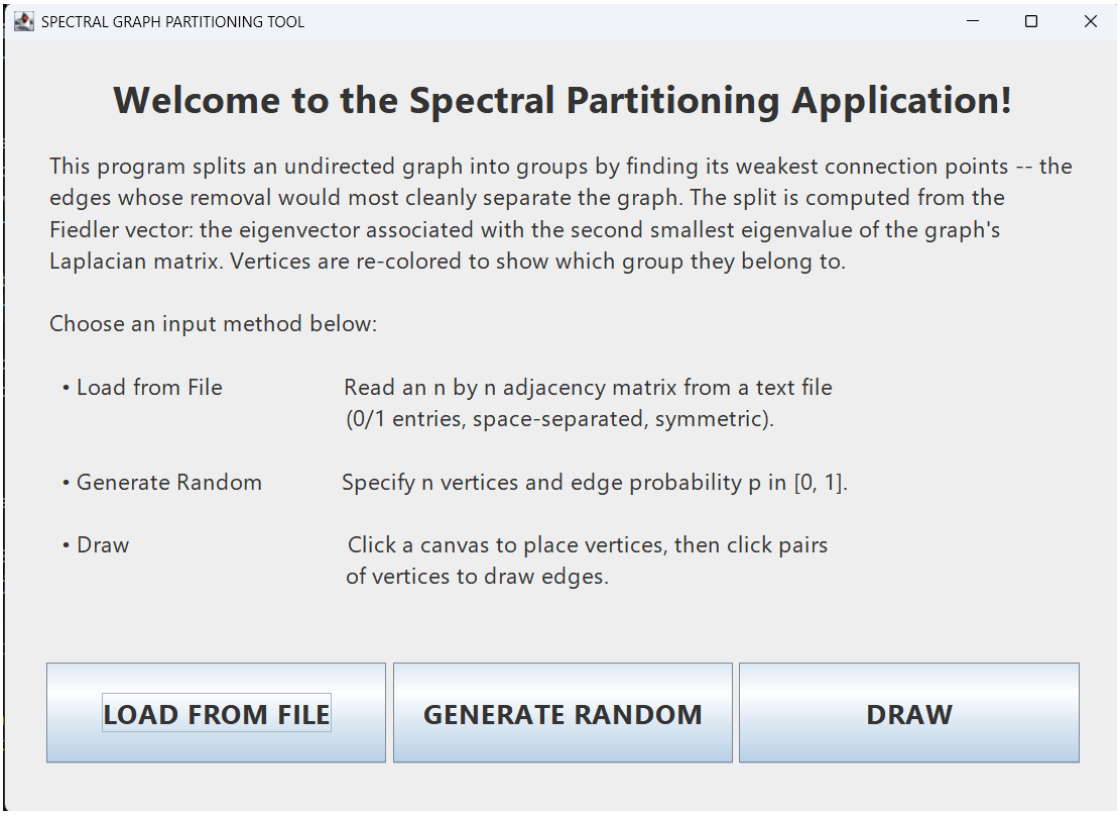


Figure 4. Screenshot from the Java Swing application showing the welcome screen with the three input modes.

In file mode, the user enters the name of a text file containing an adjacency matrix. In draw mode, the user builds a graph directly on the canvas: first by clicking empty spaces to place vertices, and then by switching to edge mode and clicking pairs of existing vertices to connect them. In random mode, the user supplies n and p, and the program generates a random graph.

5.2 Hand-drawn graph example

Draw mode is useful because the user can create a graph with an expected weak connection and then check whether the program finds it. For a first hand-drawn test, the best graph is not random. A good beginner example is two small triangle-shaped groups connected by one bridge edge.

To create this test graph, click DRAW on the welcome screen. Then click three nearby points on the left side of the canvas and three nearby points on the right side of the canvas. These six clicks create six vertices. The left three vertices will become one triangle, and the right three vertices will become another triangle.

Next, click DONE ADDING VERTICES to switch from vertex mode to edge mode. In edge mode, create edges by clicking two existing vertices one after the other. First connect the three left-side vertices to make a triangle. This means each left vertex should be connected to the other two left vertices. Then do the same thing for the three right-side vertices. Finally, connect one vertex from the left triangle to one vertex from the right triangle. This final edge is the bridge between the two groups.

After the graph is drawn, click PARTITION. This is a good first test because the expected result is easy to understand. Each triangle is strongly connected inside itself, but there is only one edge between the two triangles. A successful result should place the two triangles into different groups. The bridge edge should appear as a black cut edge, while edges inside each group should appear in light gray.

If an edge does not appear while drawing, the user should make sure the app is in edge mode and should click close to the centers of the two vertices being connected. If the user draws the two triangles but forgets the bridge edge, the graph is disconnected and the program should refuse to partition it. If the user adds several extra edges between the two triangles, the graph may no longer have a clear weak connection. In that case, a refused or unclear partition does not necessarily mean the program failed. It means the test graph no longer has the simple weak-link structure this example is meant to demonstrate.

5.3 Sample graph: two K4 cliques joined by one bridge

The supplied Sample Graph.txt file is a clean test case: two complete graphs on four vertices each, connected by one bridge edge. This is the kind of graph spectral partitioning should handle well.

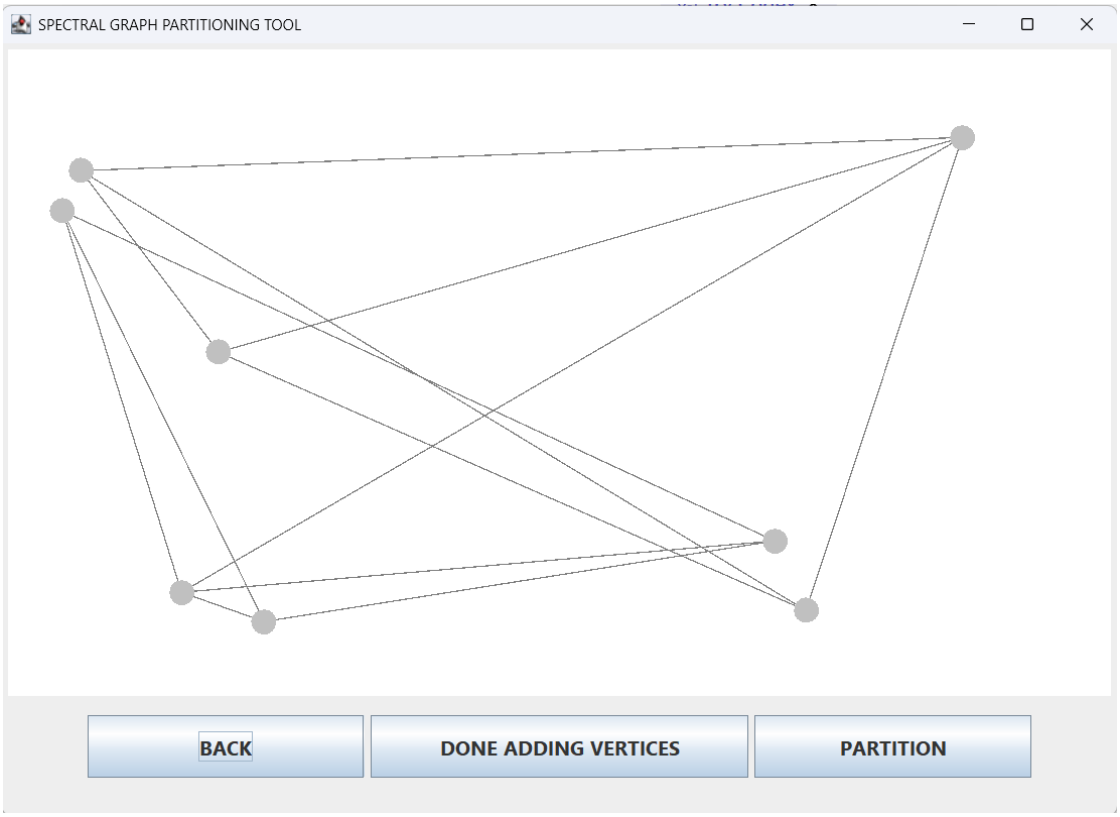


Figure 5. Screenshot from the Java Swing application showing the sample graph before partitioning. The vertices are still in their input layout.

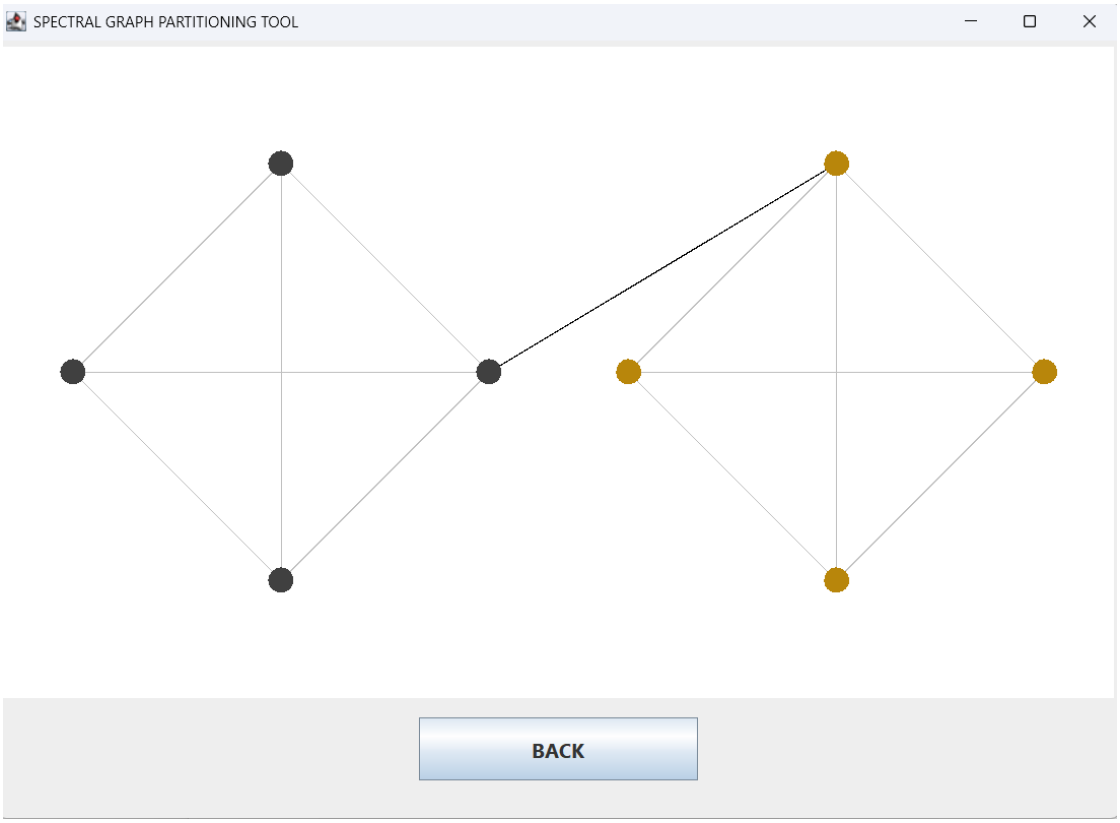


Figure 6. Screenshot from the Java Swing application showing the same graph after partitioning. The two K4 cliques are separated, and the bridge is the single black cut edge.

5.4 Random graph example

Random graphs are useful for testing because they are less controlled than the hand-drawn and file-loaded examples. However, random graphs are not guaranteed to produce a clean partition. Some are disconnected, some are too dense, and some have no obvious bottleneck.

Here, n is the number of vertices, and p is the probability that any possible edge will be included. For testing, a good starting choice for n is usually about 10 to 15 vertices with an edge probability p around 0.15. This range often gives the graph enough edges to be connected, but not so many edges that every vertex is strongly tied to every other vertex.

If the graph is disconnected, the user should generate another one. If the graph is very dense or visually tangled, the partition may be refused or may not look very meaningful. That does not necessarily mean the program failed. Random graphs are a stress test, not a guaranteed clean demo.

The example below shows an accepted random graph where the program separated a smaller group from a larger, denser group. Unlike the hand-drawn and file-loaded examples, this random graph does not have a perfect single bridge edge. It is still useful because the program produces an interpretable partition and highlights the crossing edges between the two groups.

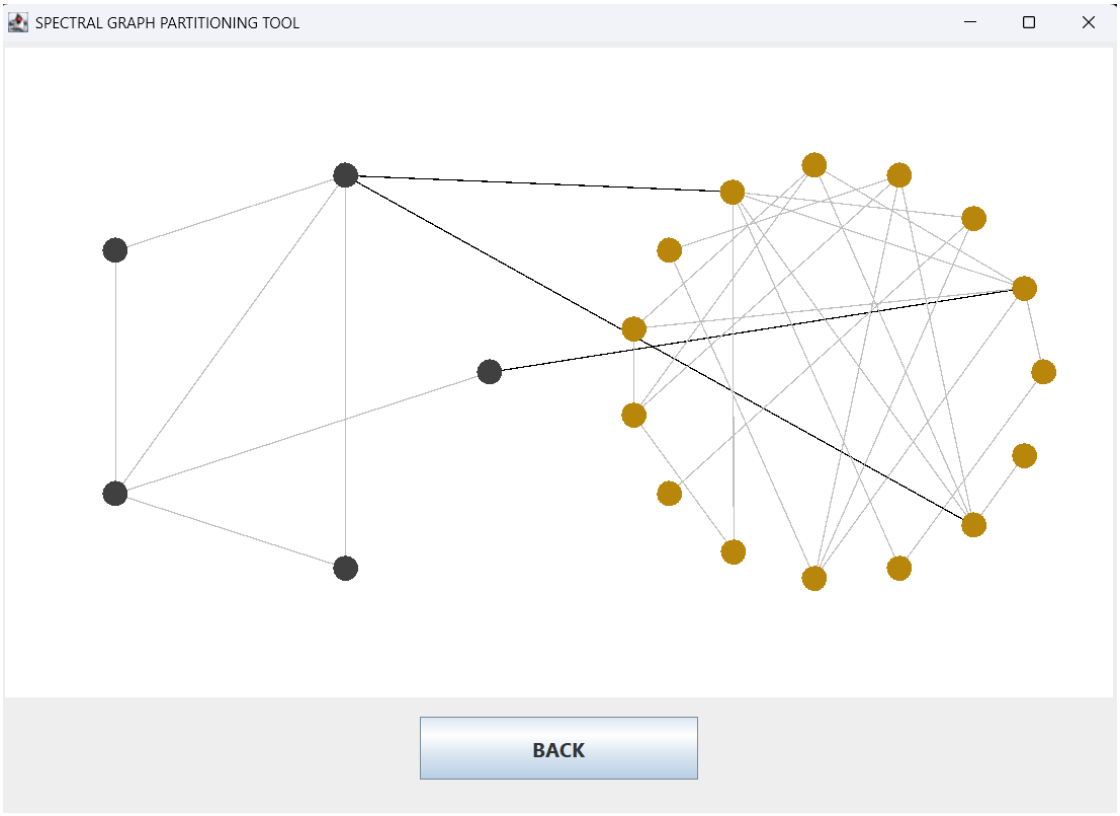


Figure 7. Screenshot from the Java Swing application showing an accepted random graph result. The program separated a smaller left-side group from a larger right-side group. The black edges show the crossing edges cut by the partition.

6. Limitations and Testing Struggles

6.1 Random graphs do not always partition

One lesson from testing is that random graphs are not guaranteed to produce a clean demo. At low p, they are often disconnected. At high p, they become dense and do not have a clear bottleneck. This is not just a program issue; it is a real property of random graphs. The refusal dialogs are there to stop the program from pretending that every graph has a meaningful clean partition. This is why the demonstration starts with hand-drawn and supplied sample graphs before showing a random graph.

6.2 The path to circular layout

The visual layout took several attempts. At first, the result screen kept vertices near their original random or hand-drawn positions. That meant a mathematically correct partition could still look like clutter. I then tried a two-region grid layout: group 0 on the left, group 1 on the right, with vertices placed into square-ish cells. That made the groups clear, but it created a new problem. Vertices in the same row or column could make edges overlap, and small structures such as triangles could become visually flattened. The final version uses a circular layout inside each group region, adapted from JGraphT's CircularLayoutAlgorithm2D. This still is not true spectral embedding, but it is a better display compromise: the groups are separated, cut edges stand out, and the internal structure cannot be flattened by repositioning.

6.3 The program only does bipartitioning

The algorithm splits the graph into two groups. Some graphs naturally have three or more clusters. In that situation, the program can only show the first two-way split. A more advanced version would recursively partition each side until no useful split remains.

6.4 Thresholds are empirical

The dense-graph threshold and cut-ratio threshold were tuned by hand. They work reasonably well for the small graphs used in this project, but the logic of choosing a particular threshold would need to be formalized for graphs beyond this toy example.

7. Future Work

The most useful extension would be recursive bipartitioning. After the first split, the program could try to split each group again. This would turn the program from a two-way partitioning tool into a k-way clustering tool.

A second extension would be true spectral embedding. A future version could store both the Fiedler vector and the third-smallest eigenvector, then use them as x and y coordinates for the result display. I originally explored this direction, but the final version uses a separate circular layout because it produced clearer results without additional mathematical and programming explorations.

Smaller future improvements include saving and reloading graphs with Java serialization, exposing the thresholds through the GUI, improving random-graph generation controls, and darkening or resizing edges for better display contrast.

8. Conclusion

What this project ultimately implements is a working spectral graph partitioning tool in Java. By the end, it does have a Swing GUI with three input modes; it has its own Graph and Matrix classes (rather than relying on a graph library); the partitioning is genuinely Fiedler-vector-based and uses a sweep procedure for picking the cut, instead of just splitting on the sign of the components; and the visualization actually pulls the two groups apart and highlights the cut edges so the result is readable. It is not a full spectral clustering package, but it does demonstrate the core idea in a concrete and interactive way.

The most important lesson was that the algorithm and the visualization have to work together. The math can find a good cut, but the app still needs to display the result clearly. The final circular layout is a practical compromise: it keeps the app simple enough for the work scope while making the result understandable to the user.

One of the goals of any theory, discovery or analysis is to create understandable and potentially meaningful, insightful and even actionable structure out of chaos. The project is simple and does not include all possible GUI features and mathematically more rigorous methods, but it does demonstrate how programming visualizations and relying on mathematics can provide meaning to a disordered clusters of data. It can serve as a starting point for more advanced modeling experiments.

References and Source Notes

The Apache Software Foundation. Apache Commons Math, version 3.6.1. https://commons.apache.org/proper/commons-math/

Fiedler, M. Algebraic connectivity of graphs. Czechoslovak Mathematical Journal, 23(2): 298-305, 1973.

JGraphT. CircularLayoutAlgorithm2D. JGraphT: a free Java graph library. https://jgrapht.org/javadoc/org.jgrapht.core/org/jgrapht/alg/drawing/CircularLayoutAlgorithm2D.html

JGraphT source code. CircularLayoutAlgorithm2D.java. https://github.com/jgrapht/jgrapht/blob/master/jgrapht-core/src/main/java/org/jgrapht/alg/drawing/CircularLayoutAlgorithm2D.java

Roch, S. Mathematical Methods in Data Science (with Python), Chapter 5, Section 5.5: Application — graph partitioning via spectral clustering. https://mmids-textbook.github.io/chap05_specgraph/05_partitioning/roch-mmids-specgraph-partitioning.html

Roch, S. MMiDS source code, utils/mmids.py, including cut_ratio and spectral_cut2. https://github.com/MMiDS-textbook/MMiDS-textbook.github.io/blob/main/utils/mmids.py

Draw.io

Tutorialspoint. How can we implement a splash screen using JWindow in Java? https://www.tutorialspoint.com/how-can-we-implement-a-splash-screen-using-jwindow-in-java

von Luxburg, U. A tutorial on spectral clustering. Statistics and Computing, 17(4): 395-416, 2007. https://arxiv.org/abs/0711.0189

Splash screen image created in Microsoft PowerPoint from Adobe Stock asset #900390074 by somneuk, licensed under Adobe Stock Standard License on 11/27/24 at 10:31 PM according to Adobe Stock license history.